# TASK 4: SENTIMENT ANALYSIS

E-commerce Review Analysis using NLP & Lexicon-based Sentiment Scoring

| | |
|---|---|
| **Project** | Sentiment Analysis |
| **Dataset** | E-commerce product review demonstration dataset |
| **Records** | 64 reviews |
| **Categories** | Electronics, Home, Beauty, Books |
| **Method** | Text preprocessing + sentiment lexicon + emotion keywords |
| **Tools** | Python • Pandas • Matplotlib |

**Important data note:** This report uses a curated demonstration set of e-commerce-style reviews to demonstrate the required NLP workflow. It is not presented as a live scrape of Amazon or social media. The same pipeline can be applied to a real CSV/API/web-scraped review dataset.

# 1. Objective & Business Questions

The goal is to classify customer text as **positive, negative or neutral**, identify emotion signals, and convert sentiment patterns into practical product and marketing insights.

## Questions asked before analysis

• What proportion of reviews are positive, neutral and negative?
• Which product categories receive the strongest sentiment?
• What emotion signals appear most often?
• Does text sentiment broadly agree with star ratings?
• What issues repeatedly appear in negative reviews?

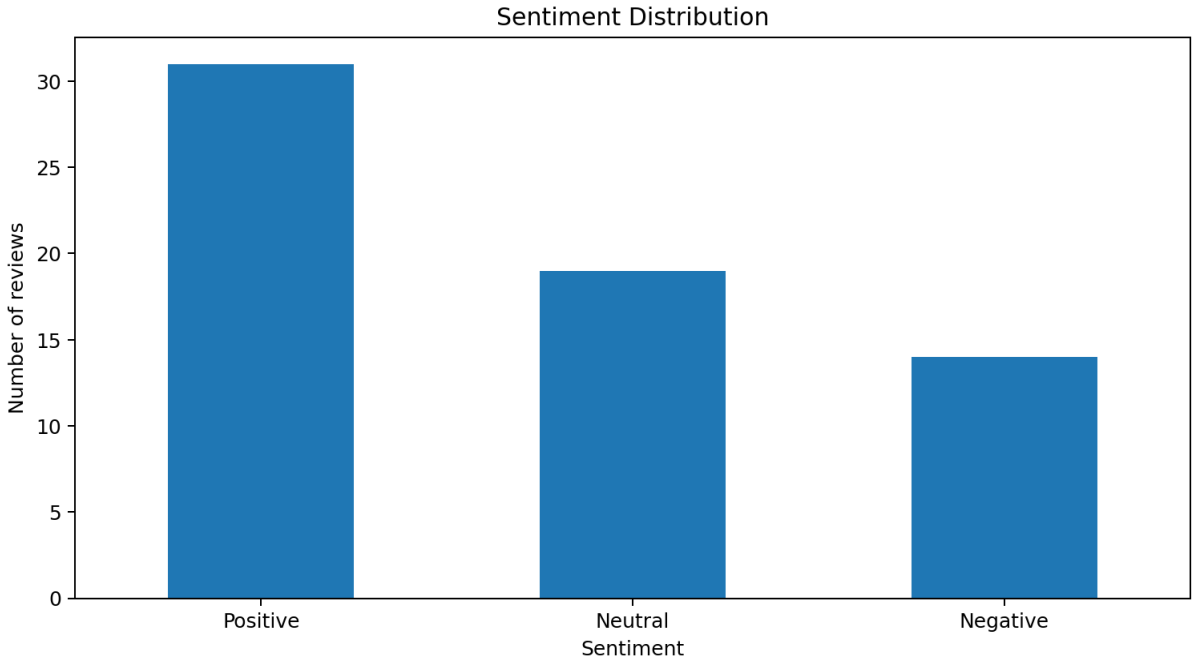


Figure 1. Overall sentiment classification.

## 2. Sentiment Classification Results

Out of 64 reviews, **31** were classified as positive, **19** as neutral and **14** as negative.

| Sentiment | Reviews | Share |
|---|---|---|
| Positive | 31 | 48.4% |
| Neutral | 19 | 29.7% |
| Negative | 14 | 21.9% |

**Interpretation:** The review set is predominantly positive, but negative reviews contain useful diagnostic information about product failures, usability and customer dissatisfaction.

# 3. Sentiment by Category

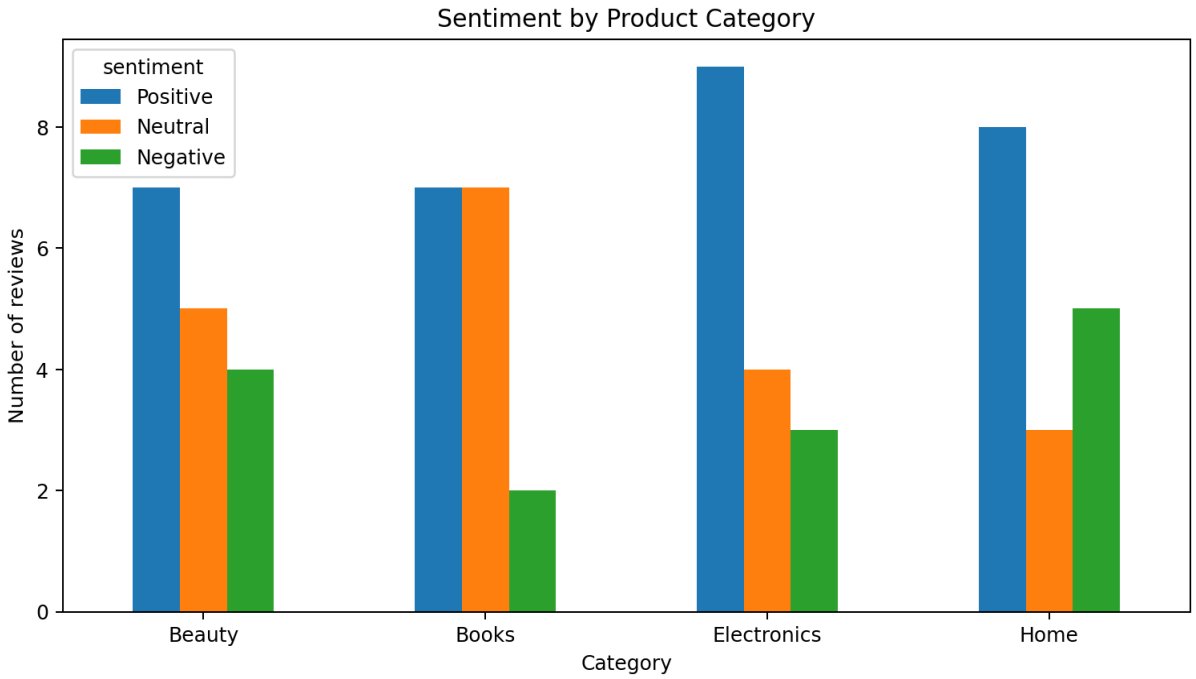


Figure 2. Distribution of sentiment across product categories.

Category comparison helps identify where customer experience is strongest or where recurring problems may need attention. Negative feedback should be read at the individual-review level before making product decisions.

# 4. Emotion Detection

A lightweight emotion lexicon was applied after tokenization. Keyword groups represent signals such as **Joy, Trust, Anger and Sadness**. This is an interpretable baseline; a production system should use a trained emotion classifier and evaluate it on labelled data.

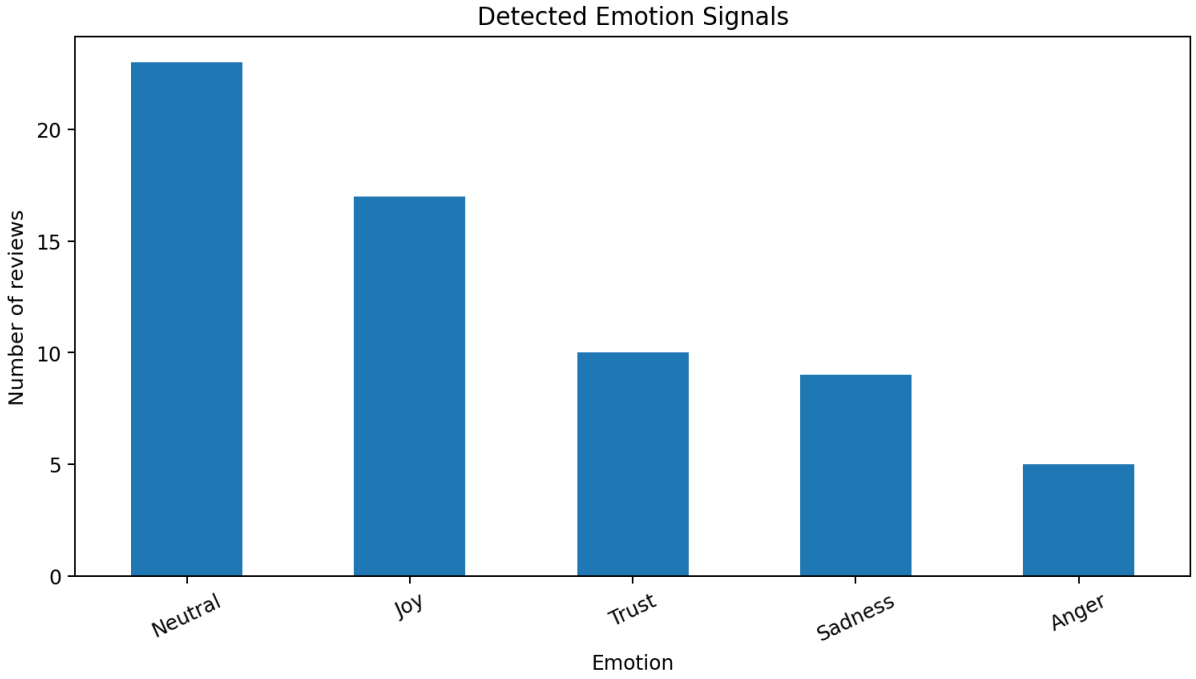


Figure 3. Emotion signals detected from review text.

# 5. NLP Methodology

**Step 1 — Text preparation:** lowercase text, remove punctuation/noise and tokenize words.

**Step 2 — Lexicon scoring:** positive and negative words receive weighted scores; simple negation handling reverses nearby sentiment.

**Step 3 — Classification:** score ≥ +1 → Positive; score ≤ −1 → Negative; otherwise → Neutral.

**Step 4 — Emotion detection:** emotion keyword groups identify dominant signals.

**Step 5 — Validation:** compare sentiment against star ratings and inspect category-level patterns.

**Why this approach?** A lexicon method is transparent and easy to explain in an internship project. For larger real-world data, it can be upgraded to VADER, TextBlob, transformer models (BERT/RoBERTa) or a supervised classifier.

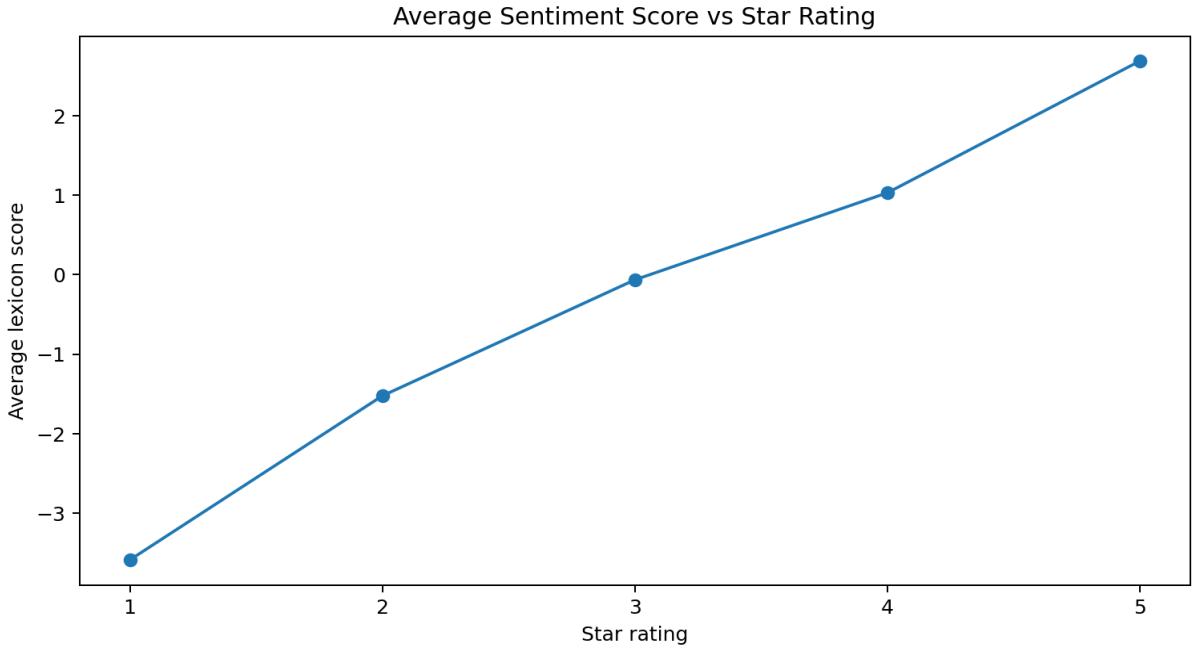


Figure 4. Sentiment score generally increases with star rating, providing a basic consistency check.

# 6. Trends & Category Insights

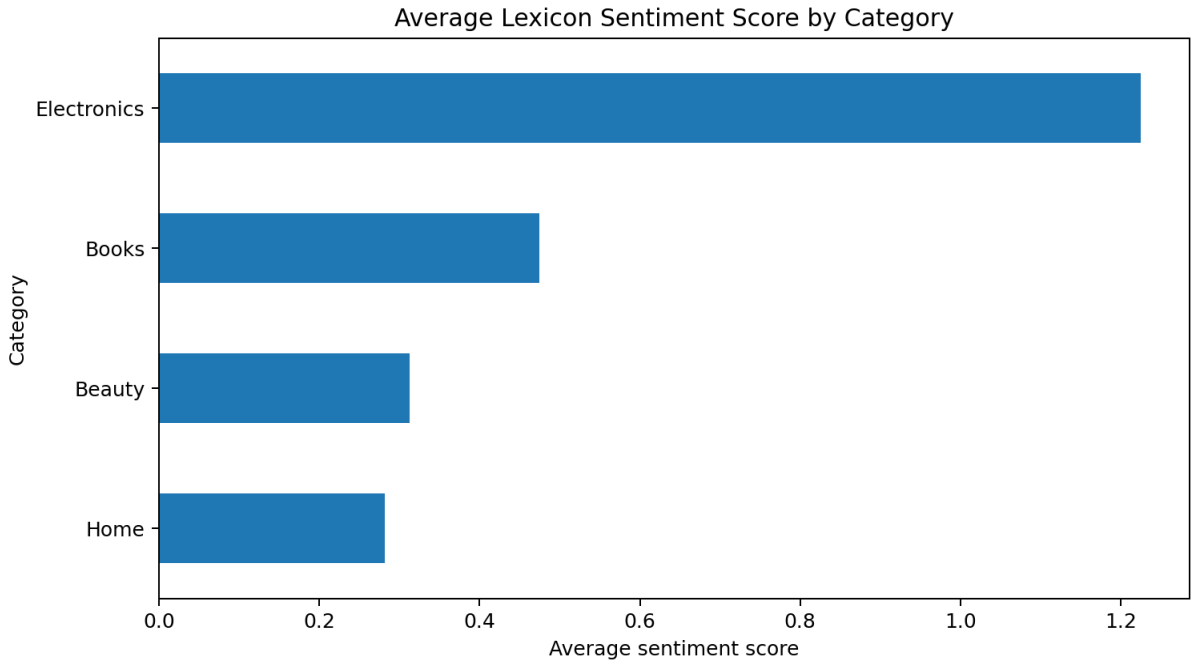


Figure 5. Average lexicon sentiment score by category.

**Insight:** Average sentiment score can be used as a compact KPI to compare categories. However, sample size and review mix should always be considered before ranking products.

# 7. Negative Feedback: Problems to Address

Negative sentiment is especially valuable because it points to specific improvement opportunities. Common themes in this dataset include reliability, product performance, usability, irritation and outdated content.

| Theme | Signal count | Recommended action |
|---|---|---|
| Usability / support | 2 | Improve onboarding, interface and support response. |
| Product performance | 2 | Investigate performance gap and improve product specs. |
| Hardware reliability | 2 | Strengthen durability testing and warranty support. |
| Build quality / leakage | 2 | Review seals, materials and manufacturing QA. |
| Stability / build quality | 1 | Run stress testing and improve construction. |
| Skin irritation | 1 | Review formulation guidance and ingredient communication. |

**Business use:** Marketing teams can track customer satisfaction, product teams can prioritize recurring complaints, and support teams can route strongly negative reviews for faster resolution.

# 8. Conclusion & Future Scope

The analysis demonstrates an end-to-end sentiment workflow: review text was processed with an interpretable sentiment lexicon, classified into positive/neutral/negative categories, enriched with emotion signals, compared across product categories and checked against ratings.

## Key takeaways

• Positive reviews form the largest group in the demonstration dataset.
• Negative reviews reveal concrete issues that can guide product improvement.
• Emotion detection adds context beyond a simple sentiment label.
• Category-level sentiment scores provide a useful monitoring KPI.
• Star-rating comparison provides a simple validation check.

## Future scope

Use a larger real-world dataset; add VADER/transformer-based sentiment; detect sarcasm and negation more deeply; perform aspect-based sentiment analysis (battery, delivery, price, quality); and build a dashboard for continuous monitoring.

**Final conclusion:** Sentiment analysis converts unstructured customer feedback into measurable signals that can support marketing, product development and customer-service decisions.

# Appendix: Reproducible Python Workflow

The following compact workflow shows the core logic used in the report and can be extended to a CSV, API or web-scraped review dataset.

```
# TASK 4 - Sentiment Analysis (core workflow)
import pandas as pd
import re

positive = {"excellent": 2.5, "amazing": 2.5, "great": 1.8,
            "good": 1.2, "love": 2.0, "happy": 1.8,
            "recommended": 1.8, "perfect": 2.4}
negative = {"terrible": -2.7, "disappointing": -2.2,
            "awful": -2.7, "frustrating": -2.0,
            "broken": -2.2, "poor": -1.5, "outdated": -1.7}

def sentiment_score(text):
    words = re.findall(r"[a-zA-Z]+", text.lower())
    score = 0
    for word in words:
        score += positive.get(word, 0)
        score += negative.get(word, 0)
    return score

def classify(score):
    if score >= 1:
        return "Positive"
    if score <= -1:
        return "Negative"
    return "Neutral"

df["score"] = df["review"].apply(sentiment_score)
df["sentiment"] = df["score"].apply(classify)

# Example analysis
print(df["sentiment"].value_counts())
print(df.groupby("category")["score"].mean())
```